Programação Linear Inteira — Projeto de Implementação

Segundo Semestre de 2026

Prof. Flávio K. Miyazawa, fkm@ic.unicamp.br

Problema de Roteamento com Prioridades e Conflitos

1 Contexto e visão geral

Uma empresa possui um depósito e precisa entregar encomendas a vários clientes. Ela dispõe de um único veículo, que faz *uma viagem por dia*: sai do depósito, visita alguns clientes e volta ao depósito. Três ingredientes tornam o problema interessante:

- **Capacidade.** O veículo tem uma capacidade máxima de carga; o peso total das encomendas de uma viagem não pode ultrapassá-la.
- **Conflitos.** Algumas encomendas não podem viajar juntas (por exemplo, alimentos e produtos químicos). Isso é descrito por um grafo de conflitos: se dois clientes são vizinhos nesse grafo, eles não podem estar na mesma viagem.
- **Prioridades.** Entregar mais cedo é melhor para alguns clientes do que para outros. Cada cliente tem um custo que depende do *dia* em que é atendido; esse custo costuma crescer com o dia, mas isso não é exigido.

Há uma tensão entre os objetivos. Juntar muitos clientes numa mesma viagem economiza deslocamento, mas a capacidade e os conflitos limitam isso. Além disso, os clientes de alta prioridade devem ser atendidos nos primeiros dias, o que pode obrigar a fazer viagens menos eficientes. O problema consiste em encontrar o melhor compromisso entre o custo de deslocamento e o custo de prioridade.

2 Notação

- Se f é uma função definida sobre um conjunto finito S e $R \subseteq S$, escrevemos $f(R) = \sum_{e \in R} f(e)$. Em particular, para o peso w temos $w(R) = \sum_{i \in R} w(i)$.
- $\mathbb{R}^+$ é o conjunto dos reais não negativos e $\mathbb{N}^* = \{1, 2, 3, \ldots\}$.
- Uma *rota* é uma *sequência* de vértices, pois a ordem de visita importa para o custo de deslocamento. Quando a ordem não importar (por exemplo, ao somar pesos ou testar conflitos), usaremos o mesmo símbolo R para o *conjunto* dos vértices da rota.

3 Dados de entrada

Uma instância do problema é dada por:

- Um grafo $G = (V, E)$, chamado **grafo de conflitos**. O conjunto V contém o depósito e os clientes. Uma aresta $\{i, j\} \in E$ significa que i e j *não podem* ser atendidos na mesma viagem.
- Um vértice especial $s \in V$, o **depósito**. Os demais vértices, $V \setminus \{s\}$, são os **clientes**.

- Uma função de custo $c : V \times V \to \mathbb{R}^+$, onde $c(i, j)$ é o custo para o veículo se deslocar diretamente de i até j. Nas instâncias deste projeto, c é simétrica ($c(i, j) = c(j, i)$) e $c(i, i) = 0$, mas não assumimos que c satisfaça a desigualdade triangular.
- Coordenadas (x_i, y_i) de cada vértice $i \in V$ no plano. Elas servem *apenas para visualização* das instâncias e das soluções; o custo de deslocamento é sempre o dado por c.
- Uma **capacidade** $C > 0$ do veículo.
- Uma função de **peso** $w : V \to \mathbb{R}^+$, onde $w(i)$ é o peso da encomenda do cliente i. Supomos $w(i) \leq C$ para todo cliente i, o que garante que toda instância tem solução viável.
- Uma função de **prioridade** $\pi : V \times \mathbb{N}^* \to \mathbb{R}^+$, onde $\pi(i, d)$ é o custo de priorização pago ao atender o cliente i no dia d. Por exemplo, $\pi(i, d) = a_i \cdot d$ modela um cliente com fator de urgência a_i que paga mais quanto mais tarde for atendido, mas qualquer tabela de valores é permitida.

4 Rotas válidas

Definição 1 (Rota válida)**.** Uma **rota** é uma sequência $R = (i_1, i_2, \ldots, i_m)$, com $m \geq 1$, de vértices *distintos*, que *não contém* o depósito s. A rota é **válida** se:

(a) *(capacidade)* $w(R) = \sum_{i \in R} w(i) \leq C$; e

(b) *(conflitos)* a rota *não* contém dois clientes i e j tais que a aresta $\{i, j\}$ pertence ao grafo de conflitos; isto é, para quaisquer $i, j \in R$ vale $\{i, j\} \notin E$.

Em outras palavras, o conjunto de clientes de uma rota válida cabe no veículo e é um *conjunto independente* de G. Fisicamente, a viagem sai de s, visita $i_1, i_2, \ldots, i_m$ nesta ordem e volta a s; o depósito não aparece na sequência porque é implicitamente o início e o fim de toda viagem.

5 Soluções viáveis e dia de entrega

Definição 2 (Solução viável)**.** Uma **solução viável** é uma lista $\mathcal{R} = (R_1, R_2, \ldots, R_k)$ de rotas válidas tal que $\{R_1, \ldots, R_k\}$ é uma *partição* de $V \setminus \{s\}$, isto é, todo cliente aparece em *exatamente uma* rota.

A *ordem* da lista tem significado: a rota R_t é percorrida no dia t. Escrevemos $t_{\mathcal{R}}(R_t) = t$ para o **dia de entrega** da rota R_t na solução $\mathcal{R}$; todos os clientes de R_t são atendidos no dia t.

Observe que:

- o número k de rotas (isto é, de dias) *não é dado*: faz parte da solução. Como cada rota é não vazia, temos $1 \leq k \leq |V| - 1$;
- duas listas com as mesmas rotas, mas em ordens diferentes, são soluções *diferentes* e têm, em geral, custos diferentes.

6 Custo de uma solução

O custo de uma solução tem duas partes: o custo de deslocamento e o custo de prioridade.

Custo logístico de uma rota. Para uma rota $R = (i_1, \ldots, i_m)$,

$$\rho(R) \;=\; c(s, i_1) \;+\; \sum_{j=2}^{m} c(i_{j-1}, i_j) \;+\; c(i_m, s).$$

É o custo de sair do depósito, percorrer os clientes na ordem dada e voltar.

Custo de prioridade de uma rota. Se todos os clientes de R são atendidos no dia d, o custo de prioridade da rota é a soma dos custos de seus clientes naquele dia:

$$\pi(R, d) \;=\; \sum_{i \in R} \pi(i, d).$$

(Usamos o mesmo símbolo π para clientes e para rotas; o contexto distingue os dois casos.)

Custo de uma solução. Para $\mathcal{R} = (R_1, \ldots, R_k)$,

$$\psi(\mathcal{R}) \;=\; \sum_{R \in \mathcal{R}} \Big(\rho(R) + \pi\big(R, t_{\mathcal{R}}(R)\big)\Big) \;=\; \sum_{t=1}^{k} \Big(\rho(R_t) + \pi(R_t, t)\Big).$$

7 O problema

Problema de Roteamento com Prioridades e Conflitos.
Entrada: uma instância $(G, s, c, (x_i, y_i)_{i \in V}, C, w, \pi)$ como descrita na seção 3.
Objetivo: encontrar uma solução viável $\mathcal{R} = (R_1, \ldots, R_k)$ de custo $\psi(\mathcal{R})$ mínimo.

8 Um exemplo completo

Considere a instância com depósito $s = 0$ e seis clientes, com capacidade $C = 10$. Neste exemplo, o custo $c(i, j)$ é a distância de Manhattan entre i e j no plano, $c(i, j) = |x_i - x_j| + |y_i - y_j|$, e a prioridade é dada por $\pi(i, d) = a_i \cdot d$. Os dados dos clientes estão na tabela abaixo; o grafo de conflitos tem as arestas $E = \big\{\{1,2\}, \{3,4\}, \{3,5\}, \{1,6\}\big\}$.

cliente i	1	2	3	4	5	6
(x_i, y_i)	$(2,0)$	$(4,0)$	$(0,3)$	$(0,5)$	$(-3,0)$	$(-3,-2)$
$w(i)$	4	3	5	2	6	3
a_i	5	1	4	1	2	1

(O depósito $s = 0$ está na origem, $(x_0, y_0) = (0, 0)$.)

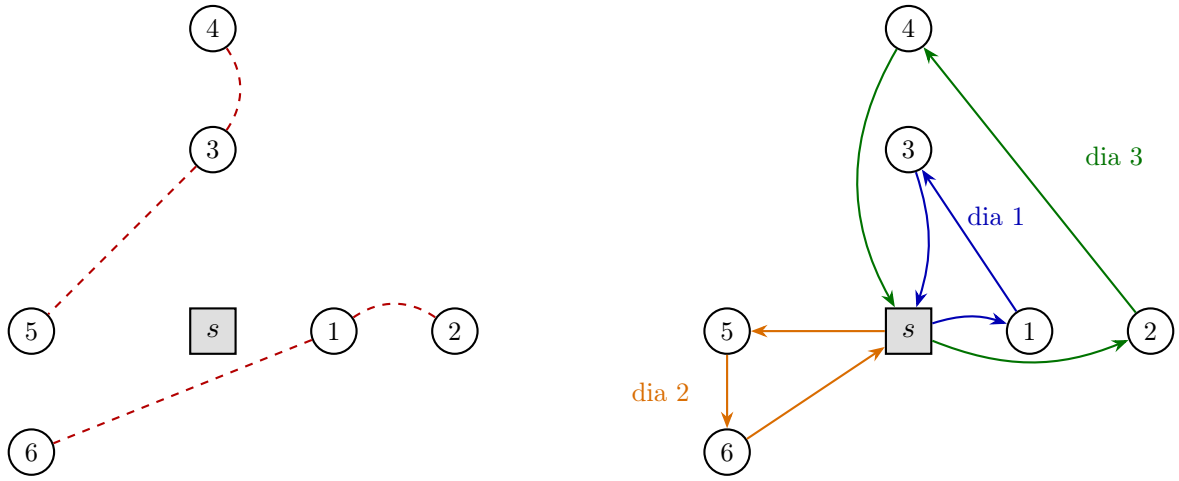
Rotas válidas e inválidas.

- $R = (1, 3)$ é válida: $w(R) = 4 + 5 = 9 \le 10$ e $\{1, 3\} \notin E$.
- $R = (1, 2)$ é inválida: o peso é $7 \le 10$, mas $\{1, 2\} \in E$ (conflito).
- $R = (2, 3, 6)$ é inválida: não há conflitos entre eles, mas $w(R) = 3 + 5 + 3 = 11 > 10$ (capacidade excedida).

Calculando o custo de uma solução. Seja $\mathcal{R} = \big((1, 3), (5, 6), (2, 4)\big)$, ou seja, $R_1 = (1, 3)$ no dia 1, $R_2 = (5, 6)$ no dia 2 e $R_3 = (2, 4)$ no dia 3. Todas as rotas são válidas ($w = 9, 9, 5$ e nenhum par em conflito) e elas particionam os clientes. Calculamos:

$$\begin{aligned}
\rho(R_1) &= c(s,1) + c(1,3) + c(3,s) = 2 + 5 + 3 = 10,\\
\rho(R_2) &= c(s,5) + c(5,6) + c(6,s) = 3 + 2 + 5 = 10,\\
\rho(R_3) &= c(s,2) + c(2,4) + c(4,s) = 4 + 9 + 5 = 18.
\end{aligned}$$

Os custos de prioridade são $\pi(R_1, 1) = (5{+}4){\cdot}1 = 9$, $\pi(R_2, 2) = (2{+}1){\cdot}2 = 6$ e $\pi(R_3, 3) = (1{+}1){\cdot}3 = 6$. Resumindo:



Instância (conflitos tracejados) Solução ótima

Figura 1: À esquerda, a instância do exemplo: o quadrado é o depósito e as arestas tracejadas são os conflitos. À direita, a solução ótima, com $k = 3$ dias. As linhas apenas indicam a ordem de visita (algumas são curvas só para não se sobreporem); o custo de deslocamento é a distância de Manhattan.

dia t	rota R_t	$w(R_t)$	$\rho(R_t)$	$\pi(R_t, t)$	subtotal
1	$(1, 3)$	9	10	9	19
2	$(5, 6)$	9	10	6	16
3	$(2, 4)$	5	18	6	24
		total	38	21	59

Logo $\psi(\mathcal{R}) = 38 + 21 = 59$. Esta é a solução ótima desta instância (pode-se conferir por enumeração).

A ordem dos dias importa. Com as *mesmas* três rotas, mas na ordem $\big((2, 4), (1, 3), (5, 6)\big)$, o custo logístico continua 38, mas o custo de prioridade passa a $(1 + 1) \cdot 1 + (5 + 4) \cdot 2 + (2 + 1) \cdot 3 = 29$, e o total é $67 > 59$: o cliente 1, que tem alta prioridade, é atendido mais tarde.

9 Observações para pensar

As observações abaixo não fazem parte da definição do problema, mas ajudam a entendê-lo e a modelá-lo.

1. **Duas decisões separáveis.** O custo logístico $\rho(R)$ depende *apenas* da sequência de visita dentro da rota, e não do dia; já o custo de prioridade $\pi(R, d)$ depende *apenas* do conjunto de clientes da rota e do dia, e não da ordem de visita. Assim, a solução envolve três decisões: (i) como particionar os clientes em rotas válidas; (ii) em que ordem visitar os clientes de cada rota; (iii) em que dia executar cada rota.

2. **Subproblemas conhecidos.** Fixados os conjuntos de clientes de cada rota, escolher a ordem de visita em cada uma é um problema de caixeiro-viajante (ciclo mínimo passando por s e pelos clientes da rota), e escolher os dias é um problema de *atribuição* entre rotas e dias, com custo $\pi(R, d)$ para atribuir a rota R ao dia d.

3. **Casos particulares.** Se $E = \emptyset$ e $\pi \equiv 0$, temos o problema de roteamento de veículos capacitado com frota ilimitada. Se $c \equiv 0$ e $\pi \equiv 0$, temos um problema de empacotamento (*bin packing*) com

conflitos. Se E é completo (todos os pares de clientes em conflito), cada rota atende um único cliente e resta apenas escolher a ordem dos clientes, isto é, o dia de cada um.

4. **Número de dias.** Como $k \leq |V| - 1$, é possível limitar superiormente o número de dias ao modelar o problema.

10 Convenções e formato dos arquivos

10.1 Convenções

- Se o grafo de conflitos tem n vértices, então $V = \{0, 1, 2, \ldots, n-1\}$.
- O depósito $s \in V$ é informado como parâmetro do programa (e não no arquivo de entrada). Os clientes são os vértices de $V \setminus \{s\}$.
- Uma aresta do grafo de conflitos é sempre escrita como um par $i\,j$ com $i < j$, e significa que i e j *não podem* estar na mesma rota.
- O arquivo traz o custo $c(i, j)$ apenas para $i < j$: os custos são *simétricos*, $c(i, j) = c(j, i)$, e $c(i, i) = 0$.
- Os valores $w(s)$ e $\pi(s, \cdot)$ do depósito aparecem no arquivo, mas são ignorados.

10.2 Arquivo de entrada

O arquivo de entrada é um arquivo texto. Linhas que começam com o caractere `#` são comentários e devem ser ignoradas. As demais linhas aparecem nesta ordem:

1. **Uma linha** com o número n de vértices e o número m de arestas do grafo de conflitos: `<n> <m>`.
2. n **linhas**, uma por vértice, em ordem crescente de i. A linha do vértice i tem $n + 2$ inteiros:

   `<w_i> <x_i> <y_i> <pi(i,1)> <pi(i,2)> ... <pi(i,n-1)>`

   onde $w_i = w(i)$ é o peso, (x_i, y_i) são as coordenadas no plano (apenas para visualização) e $\pi(i, 1), \ldots, \pi(i, n-1)$ são os custos de prioridade de i em cada dia $1, \ldots, n-1$.
3. $n(n-1)/2$ **linhas**, uma para cada par de vértices $i < j$:

   `<i> <j> <c(i,j)>`

   onde $c(i, j) \geq 0$ é o custo de deslocamento entre i e j.
4. m **linhas**, uma para cada aresta do grafo de conflitos:

   `<i> <j>` com $i < j$.

Exemplo. A instância da Seção 8 (depósito $s = 0$, clientes $1, \ldots, 6$) corresponde ao arquivo abaixo, com $\pi(i, d) = a_i \cdot d$. Ele deve ser executado com $s = 0$ e $C = 10$.

```
# numero de vertices e numero de arestas do grafo de conflitos
7 4
# vertices: w x y pi(i,1) ... pi(i,6)   (o vertice 0 e o deposito)
0 0 0 0 0 0 0 0 0
4 2 0 5 10 15 20 25 30
3 4 0 1 2 3 4 5 6
5 0 3 4 8 12 16 20 24
2 0 5 1 2 3 4 5 6
```

```
6 -3 0 2 4 6 8 10 12
3 -3 -2 1 2 3 4 5 6
# custos de deslocamento: i j c(i,j), com i<j
0 1 2
0 2 4
0 3 3
0 4 5
0 5 3
0 6 5
1 2 2
1 3 5
1 4 7
1 5 5
1 6 7
2 3 7
2 4 9
2 5 7
2 6 9
3 4 2
3 5 6
3 6 8
4 5 8
4 6 10
5 6 2
# arestas do grafo de conflitos: i j, com i<j
1 2
3 4
3 5
1 6
```

10.3 Arquivo de saída

O programa grava a solução encontrada em um arquivo texto, também com comentários iniciados por #. Ele contém, nesta ordem:

1. o valor $\psi(\mathcal{R})$ da melhor solução encontrada;
2. o melhor limitante inferior para o valor ótimo, obtido pelo algoritmo;
3. o número k de rotas (dias);
4. k linhas: a t-ésima linha contém os clientes da rota R_t, *na ordem de visita* (o depósito não aparece).

Para a instância do exemplo, a saída da solução ótima é:

```
# valor da solucao
59
# limitante inferior
59
# numero de rotas (dias)
3
# rota de cada dia, na ordem de visita
1 3
5 6
2 4
```

11 Projeto 1: Algoritmo *Branch-and-Price*

11.1 O que deve ser feito

Neste primeiro projeto, vocês devem implementar um algoritmo exato de *branch-and-price* para o Problema de Roteamento com Prioridades e Conflitos. Os requisitos são:

1. **Formulação.** Usar necessariamente variáveis binárias $x_{R,d}$, onde R é uma rota válida e d é o dia em que a rota R é realizada (Seção **??**).

2. **Geração de colunas.** O problema mestre deve ser resolvido por geração de colunas (Seção **??**). Cada coluna nova deve ser obtida assim:

   - *primeiro*, por um método heurístico;
   - *somente se* a heurística não encontrar nenhuma coluna candidata, por um método exato. Ele pode ser um método de enumeração, uma formulação de programação linear inteira, ou ambos (versão híbrida).

3. **Ramificação.** A árvore de enumeração deve usar o *branching de Ryan–Foster*, visto em sala (Seção **??**).

4. **Uso do Gurobi.** No algoritmo de branch-and-price em si (relaxações dos nós, limitantes inferiores e podas), o Gurobi deve ser usado *apenas como solver de programação linear*: a árvore de enumeração, o branching e a geração de colunas são implementados por vocês. Há duas exceções, em que o Gurobi pode resolver problemas com variáveis inteiras:

   - no problema de pricing exato, caso vocês optem por uma formulação de programação linear inteira;
   - na *heurística de arredondamento* (parâmetro `-h`), descrita na Seção **??**.

5. **Implementação e execução.** O programa deve ser escrito em **C++** e receber os parâmetros abaixo na linha de comando:

`-f <arquivo>`	arquivo de entrada (Seção 10);
`-s <vértice>`	vértice que será o depósito;
`-C <inteiro>`	capacidade do veículo (valor inteiro);
`-t <segundos>`	tempo máximo de execução, em segundos;
`-h <segundos>`	*(opcional)* tempo adicional, em segundos, além de `-t`, para a heurística de arredondamento (Seção **??**);
`-o <arquivo>`	arquivo onde a saída será gravada (Seção 10.3).

   Por exemplo: `./roteamento -f inst.txt -s 0 -C 10 -t 60 -h 5 -o saida.txt`. Ao esgotar o tempo `-t`, o programa executa a heurística de arredondamento (se `-h` foi dado), grava a melhor solução viável encontrada e o melhor limitante inferior obtido, e termina. O tempo total de execução é de, no máximo, `-t` mais `-h` segundos (mais o tempo de gravar a saída).

Hipótese sobre a prioridade. Neste projeto supomos que a função π *não decresce com o dia*:

$$\pi(i,d) \;\le\; \pi(i,d+1) \qquad \text{para todo } i \in V \setminus \{s\},\ d \ge 1.$$

(Versões posteriores do projeto poderão usar funções π mais gerais.)

12 Entrega, relatório e uso de ferramentas de IA

O projeto é **individual**. O objetivo é que cada aluno *domine* o algoritmo que entrega: entender a formulação, o pricing, o branching e o código, e saber verificar que tudo funciona.

12.1 Uso de ferramentas de IA

O uso de ferramentas de IA (por exemplo, ChatGPT e Claude) é **permitido**, sob as condições abaixo.

- **Transparência.** Todas as *conversas* com ferramentas de IA devem ser entregues completas, em ordem cronológica, com os prompts e as respostas (e não apenas os prompts). Um prompt isolado não mostra o que a ferramenta respondeu nem o que foi aceito ou rejeitado.
- **Responsabilidade.** Você é responsável por todo o código e todo o texto entregues, como se fossem seus. Você deve conseguir explicar e modificar qualquer parte deles (veja a Seção 12.6).
- **Individualidade.** Não é permitido copiar código ou texto de outros alunos, nem compartilhar o seu. Isso inclui as suas conversas com ferramentas de IA.

12.2 O que entregar

A entrega é feita **por e-mail**, para `fkm@unicamp.br`, com o assunto (*subject*) exatamente igual a

`[PLI] Roteamento com Prioridades e Conflitos - Branch & Price`

e com um único arquivo compactado (`.zip` ou `.tar.gz`) anexado, com a estrutura:

```
codigo/        fontes em C++ e Makefile (obrigatorio)
testes/        verificador de solucoes, instancias de teste e scripts
ia/            conversas completas com ferramentas de IA, em ordem cronologica
relatorio.pdf  relatorio (descrito abaixo)
README         como compilar e executar, versoes usadas
```

Além disso, o histórico do repositório `git` (com *commits* feitos ao longo do desenvolvimento) deve ser entregue junto com o código.

Regras sobre os arquivos.

- O arquivo enviado **não deve conter código executável nem código objeto**: nada de executáveis, arquivos `.o`, bibliotecas (`.a`, `.so`), nem diretórios de compilação. Envie apenas o código-fonte, os scripts e os demais arquivos de texto, PDFs e instâncias de teste.
- A entrega **deve ter um** `Makefile` em `codigo/`: o comando `make`, executado nesse diretório, deve compilar o programa e gerar o executável. O caminho de instalação do Gurobi deve ser configurável por uma variável no início do `Makefile`.

Prazo. A entrega deve ser feita até o dia **15/Novembro/2026**.

12.3 Relatório

O relatório deve conter as seções seguintes.

1. **Descrição do algoritmo**, em suas próprias palavras: a formulação do mestre; o problema de pricing; as colunas iniciais; a heurística de pricing e o método exato (e, no caso híbrido, como eles se combinam); a regra de escolha do par de Ryan–Foster; a estratégia de busca na árvore; o tratamento da viabilidade do PMR em cada nó; as estruturas de dados principais.
2. **Uma iteração calculada à mão.** Na instância do exemplo da Seção 8, mostre as variáveis duais u_i e λ_d de uma iteração da geração de colunas, o custo reduzido de pelo menos duas colunas candidatas e, em um nó fracionário, o par escolhido para o branching, com os valores de φ. Compare com o que o seu programa imprime.

3. **Experimentos** (Seção 12.5).

4. **Onde a IA errou.** Descreva pelo menos dois casos concretos em que uma resposta de uma ferramenta de IA estava errada, ineficiente ou incompleta: o que ela propôs, como você percebeu o problema e como o corrigiu. Se não usou IA, escreva que não usou, e descreva dois erros seus que foram difíceis de achar.

5. **Limitações.** Em que tipos de instância o seu algoritmo se comporta mal, e por quê.

6. Ao esgotar o tempo `-t` sem prova de otimalidade, se `-h` foi dado, execute a heurística de arredondamento (Seção **??**) por até h segundos adicionais, atualize a incumbente e grave a saída.

12.4 Verificação e testes

A entrega deve incluir:

- Um **verificador de soluções** escrito por você, que lê o arquivo de entrada e o arquivo de saída e confere: cada cliente aparece em exatamente uma rota; cada rota respeita a capacidade e os conflitos; o custo ψ informado confere com o custo recalculado; o limitante inferior informado é menor ou igual ao custo da solução.

- Um algoritmo de **enumeração completa**, independente do branch-and-price, usado para comparar os valores ótimos em instâncias pequenas (por exemplo, $n \leq 8$), com pelo menos algumas dezenas de instâncias geradas aleatoriamente.

- A instância do exemplo da Seção 8 como teste obrigatório (valor ótimo 59).

- Um **log de execução** impresso na saída de erro (`stderr`), com, por nó da árvore: limitante inferior, melhor solução conhecida, número de colunas, número de chamadas de pricing heurístico e exato, e tempo gasto.

12.5 Experimentos

Use as instâncias fornecidas pelo professor (e, se quiser, outras geradas por você). Reporte, por instância, o tempo, o número de nós da árvore, o número de colunas geradas, o limitante inferior e a melhor solução obtidos e o gap final. Faça comparações (*ablações*) entre, no mínimo:

- pricing só exato *versus* heurística seguida de exato;

- com e sem o pré-processamento dos nós (classes de clientes que ficam juntos e propagação entre as decisões de branching);

- duas regras de escolha do par para ramificar;

- duas estratégias de busca na árvore (por exemplo, profundidade e melhor limitante);

- com e sem a heurística de arredondamento (`-h`).

Apresente tabelas e pelo menos um gráfico, e **discuta** os resultados: o que os dados mostram e por quê. Inclua os *scripts* que reproduzem os experimentos.

12.6 Ponto de controle e entrevista final

- **Ponto de controle.** Antes da entrega final, será pedido um ponto de controle com a geração de colunas funcionando no nó raiz (sem branching), com o limitante inferior da relaxação linear reportado. O prazo será informado em sala.

- **Entrevista individual no fim do curso.** A avaliação inclui uma **entrevista** com cada aluno, no fim do curso (a data será informada em sala). Nela, o aluno deve conseguir *explicar o seu código e o seu funcionamento, e as teorias envolvidas na implementação.*

Na entrevista, o aluno deve ser capaz de explicar, por exemplo:

- os **modelos**: a formulação do problema mestre e o significado de cada família de restrições;
- **como se obtém o custo reduzido** de uma coluna, e o papel das variáveis duais u_i e λ_d;
- **como chegar à formulação do problema de pricing**, a partir do dual do problema mestre restrito, e como ela muda a cada nó da árvore por causa do branching;
- **como foi feita a heurística** de pricing e **como foi feito o algoritmo exato**, e por que as escolhas foram essas;
- por que o limitante inferior só é válido depois que o método exato prova que não há coluna de custo reduzido negativo;
- a heurística de arredondamento: como e com que colunas é chamada, e por que seu resultado só vale como limitante superior;
- o branching de Ryan–Foster: como se escolhe o par, e como cada ramo é imposto no mestre e no pricing;
- as **estruturas de dados** adotadas e por quê;
- os **cuidados** tomados na implementação (por exemplo, a viabilidade do mestre restrito em cada nó, colunas duplicadas, erros numéricos).

O aluno precisa ter o **controle do algoritmo**, em alto nível e em seus detalhes: saber o que o seu programa faz e por quê, e não apenas que ele funciona. A entrevista pode incluir uma pequena alteração no código feita ao vivo.

Sobre a especificação. Este enunciado descreve o problema, a formulação e o esquema geral do algoritmo de forma suficiente para que o programa possa ser construído, mas *não* diz como codificá-lo. As decisões de projeto (estruturas de dados, heurísticas, método exato, regras de escolha e de busca) são do aluno, que deve estar no controle do algoritmo e saber justificar cada uma delas. Código que o aluno não consegue explicar na entrevista não será considerado como compreendido.

12.7 Avaliação

A avaliação considerará:

- a correção e a completude do algoritmo, testadas também em *instâncias não divulgadas* previamente;
- a qualidade da verificação (verificador, enumeração, testes);
- a qualidade dos experimentos e da discussão;
- a clareza e a honestidade do relatório, incluindo a seção sobre a IA;
- a compreensão demonstrada na entrevista.

Entregar código que roda sem demonstrar compreensão na entrevista **não** garante boa nota.